

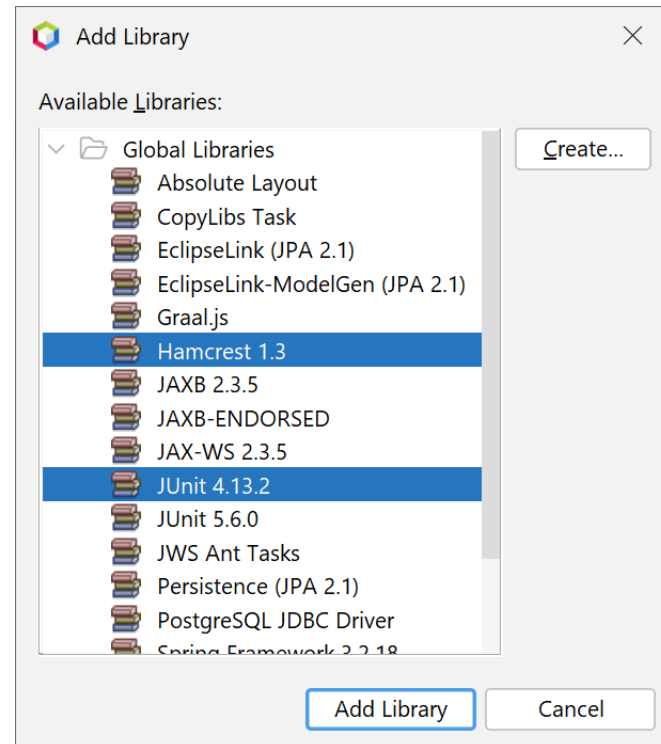
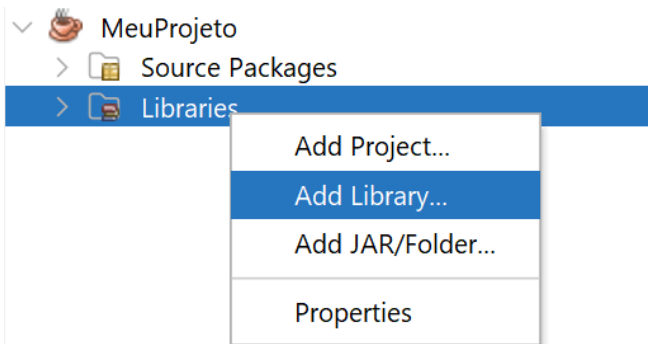
Testes de unidade com JUnit

Sobre o JUnit

- JUnit é um framework para execução de teste de unidade em Java
- Criado em 1997 por Erich Gamma e Kent Beck
- Hospedado no Source Forge
- Distribuído sob a licença IBM Common Public Liscence
- Modelo adotado por outras linguagens

Como testar com Junit no NetBeans

- Para habilitar a execução de testes, clicar com o botão direito em “Libraries” e selecionar “Add Library”
- Selecionar as bibliotecas:
 - Hamcrest 1.3 e
 - JUnit 4.13.2



Testes com JUnit

- Como vamos implementar testes para a camada de negócio, haverá uma classe “paralela” (de teste) para cada classe a ser testada
- Em geral, o nome desta classe é igual ao nome da classe a ser testada, porém com sufixo “Test”
- Para cada “caso de teste”, criar um método e introduzir a anotação @Test
 - O método deve ser público e não pode ter parâmetros
 - O método não pode retornar dados (void)
 - Dentro deste método, deve-se utilizar um comando *assert*, para validar uma situação

Como testar com JUnit

- Exemplo:

```
1 import static org.junit.Assert.*;
2 import org.junit.*;
3
4 public class CalculadoraTest {
5
6     @Test
7     public void test1_SomarNumeros() {
8         Calculadora calc = new Calculadora();
9         int resultado = calc.somar(10,75);
10        assertEquals(85, resultado);
11    }
12
13 }
```

↑ Valor esperado ↓ Valor obtido

“Espero que o somar()
resulte em 85”

Métodos Assert

- Os métodos *assert* são utilizados para verificar se uma condição é verdadeira. Caso a verificação não seja bem sucedida, é lançada uma exceção

Método	Descrição
<code>assertEquals(long esperado, long real)</code>	Verifica se os dois números inteiros são iguais
<code>assertEquals(double esperado, double real, double limite)</code>	Verifica se dois números decimais são iguais, sem atingir o limite de diferença
<code>assertEquals(objetoEsperado, objetoReal)</code>	Verifica se dois objetos são iguais
<code>assertTrue(boolean condição)</code>	Verifica se a condição é verdadeira
<code>assertFalse(boolean condição)</code>	Verifica se a condição é falsa
<code>assertNotNull(objeto)</code>	Verifica se a variável referencia um objeto
<code>assertNull(objeto)</code>	Verifica se a variável não referencia um objeto

Casos de teste com o mesmo contexto

- Quando dois ou mais casos de teste compartilham o mesmo contexto, é possível definir o contexto num método que contém a anotação *@Before*

```
@Before
public void inicializarContexto() {
    calc = new Calculadora();
}
```

- Igualmente, é possível definir um método para ser executado após cada caso de teste, com a anotação *@After*

```
@After
public void finalizarContexto() {
    arquivo.close();
}
```

Exceções

- A anotação @Test permite utilizar o parâmetro “expected” para implementar casos de teste que esperam que ocorra uma determinada exceção

```
1 @Test(expected=IllegalArgumentException.class)
2 public void test013() throws Exception {
3     imp.setSalario(-100);
4 }
```

Tempo limite de execução

- Para limitar o tempo máximo de execução de um teste, pode ser utilizado o parâmetro *timeout*

```
1 @Test(timeout = 1000)
2 public void testLoadUserCfg() {
3     db.setUser("teste");
4     db.loadUserConfig();
5 }
```